2 链表
2.1 理论基础
2.1.1 存储方式
2.1.2 链表操作
2.2 经典题目
2.2.1 链表操作
例题：leetcode 203 - 移除链表元素
例题：leetcode 707 - 设计链表
2.2.2 方向改变
例题：leetcode 206 - 反转链表
例题：leetcode 24 - 两两交换链表中的节点
例题：leetcode 19 - 删除链表的倒数第 N 个结点
2.2.3 双指针与链表
例题：面试题 02.07 - 链表相交
例题：leetcode 142 - 环形链表 II

2 链表

- 考察链表的操作其实就是考察指针的操作，是面试中的常见类型。

2.1 理论基础

- 链表是一种通过指针串联在一起的线性结构，每一个节点由两部分组成，一个是数据域一个是指针域（存放指向下一个节点的指针），最后一个节点的指针域指向null（空指针的意思）。
    - 链表的入口节点称为链表的头结点也就是head。
- 双链表：每一个节点有两个指针域，一个指向下一个节点，一个指向上一个节点。
    - 所以双链表既可以向前查询也可以向后查询。
- 循环链表，顾名思义，就是链表首尾相连。
    - 循环链表可以用来解决约瑟夫环问题。

2.1.1 存储方式

- 数组是在内存中是连续分布的，但是链表在内存中可不是连续分布的。
- 链表是通过指针域的指针链接在内存中各个节点。
- 所以链表中的节点在内存中不是连续分布的 ，而是散乱分布在内存中的某地址上，分配机制取决于操作系统的内存管理。

2.1.2 链表操作

- 定义链表

```
// 单链表
struct ListNode {
    int val;  // 节点上存储的元素
    ListNode *next;  // 指向下一个节点的指针
    ListNode(int x) : val(x), next(NULL) {}  // 节点的构造函数
};
```

- 创建节点

```
ListNode* node1 = new ListNode(5); // 创建值为5的节点，next为NULL
```

- 链接节点

```
ListNode* node2 = new ListNode(10);
node1->next = node2; // node1的next指向node2
```

- 遍历链表

```
ListNode* current = head;
while (current != NULL) {
    // 处理 current->val
    current = current->next;
}
```

- 特性：删除添加某个节点时，只需改变前一项指针指向，并释放删除项内存，并不需要移动位置，所以计算量很小。

2.2 经典题目

2.2.1 链表操作

例题：leetcode 203 - 移除链表元素

[203. 移除链表元素]

- 给你一个链表的头节点 `head` 和一个整数 `val` ，请你删除链表中所有满足 `Node.val == val` 的节点，并返回 **新的头节点** 。

```
class Solution {
public:
    ListNode* removeElements(ListNode* head, int val) {
        while(head!=nullptr && head->val==val){
            ListNode *temp = head;
            head = temp -> next;
            delete temp;
        }

        ListNode *current = head;
        while(current!=nullptr && current->next!=nullptr){
```

```
            if (current->next->val == val) {
                ListNode* tmp = current->next;
                current->next = tmp->next;
                delete tmp;
            }
            else current = current->next;
        }
        return head;
    }
};
```

- 基本思路：查询next节点符不符合要求，符合则删除，再将该节点接入到下下个节点上。
    - 这样会疏漏头节点的筛查，于是需要再最初做一次判断，确保保留头节点为第一个不符合条件的节点。
- 重点：遍历链表的写法。

例题：leetcode 707 - 设计链表

[707. 设计链表]

- 你可以选择使用单链表或者双链表，设计并实现自己的链表。
- 单链表中的节点应该具备两个属性：`val` 和 `next` 。`val` 是当前节点的值，`next` 是指向下一个节点的指针/引用。
- 如果是双向链表，则还需要属性 `prev` 以指示链表中的上一个节点。假设链表中的所有节点下标从 **0** 开始。
- 实现 `MyLinkedList` 类：
    - `MyLinkedList()` 初始化 `MyLinkedList` 对象。
    - `int get(int index)` 获取链表中下标为 `index` 的节点的值。如果下标无效，则返回 `-1`。
    - `void addAtHead(int val)` 将一个值为 `val` 的节点插入到链表中第一个元素之前。在插入完成后，新节点会成为链表的第一个节点。
    - `void addAtTail(int val)` 将一个值为 `val` 的节点追加到链表中作为链表的最后一个元素。
    - `void addAtIndex(int index, int val)` 将一个值为 `val` 的节点插入到链表中下标为 `index` 的节点之前。如果 `index` 等于链表的长度，那么该节点会被追加到链表的末尾。如果 `index` 比长度更大，该节点将 **不会插入** 到链表中。
    - `void deleteAtIndex(int index)` 如果下标有效，则删除链表中下标为 `index` 的节点。

```
class MyLinkedList {
private:
    struct ListNode{
            int val;
            ListNode* next;
            ListNode(int x): val(x),next(nullptr) {}
        };

    ListNode* fakehead;
```

```
    int len;
public:
    MyLinkedList() {
        fakehead = new ListNode(0);
        len = 0;
    }
```

- 对于这道题目，我们使用到假头节点的方法，我们在初始化链表时，初始化一个假的头节点，但不计入链表长度，好处是对于单向链表的真头节点，向前插入会特别方便。
    - 注意初始化的时候 `new` 不能省去。

```
int get(int index) {
    if(index > len - 1) return -1;
    ListNode* cur = fakehead;
    while(index >= 0){
        cur=cur->next;
        index--;
    }
    return cur->val;
}

void addAtHead(int val) {
    ListNode* node = new ListNode(val);
    node->next=fakehead->next;
    fakehead->next = node;
    len++;
}

void addAtTail(int val) {
    ListNode* cur = fakehead;
    while(cur->next!=nullptr) cur=cur->next;
    ListNode* node1 = new ListNode(val);
    cur->next = node1;
    len++;
}

void addAtIndex(int index, int val) {
    if(index==len) addAtTail(val);
    else if(index<len){
        ListNode* cur = fakehead;
        while(index>0){
            index--;
            cur = cur->next;
        }
        ListNode* node2 = new ListNode(val);
        node2->next = cur->next;
        cur->next = node2;
        len++;
    }
}
```

```

void deleteAtIndex(int index) {
    if(index<=len-1){
        ListNode* cur = fakehead;
        while(index>0){
            index--;
            cur = cur->next;
        }
        ListNode* tmp = cur->next;
        cur->next = cur->next->next;
        delete tmp;
        len--;
    }
}
};
```

- 注意到假头节点并不属于头节点，所以操作的时候要向后移一位。

2.2.2 方向改变

例题：leetcode 206 - 反转链表

[206. 反转链表]

- 给你单链表的头节点 `head` ，请你反转链表，并返回反转后的链表。

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode* cur = head;
        ListNode* pre = nullptr;
        while(cur!=nullptr){
            ListNode* tmp = cur->next;
            cur->next = pre;
            pre = cur;
            cur = tmp;
        }
        return pre;
    }
};
```

- 循环思路：对于每个节点，把指向改成上个节点：
    - 那么我们需要一个变量去存储上个节点，也需要一个节点去存储下个节点来保证我们改变指向之后能正确地延续循环。

```
class Solution {
public:
    ListNode* reverse(ListNode* pre, ListNode* cur){
        if(cur==nullptr) return pre;
        ListNode* tmp = cur->next;
        cur->next = pre;
```

```
        pre = cur;
        cur = tmp;
        return reverse(pre,cur);
    }
    ListNode* reverseList(ListNode* head) {
        return reverse(nullptr,head);
    }
};
```

- 递归思路：类似于上述的循环思路，我们保证不停循环直到节点为空，然后输出上一个节点作为头节点即可。

例题：leetcode 24 - 两两交换链表中的节点

[24. 两两交换链表中的节点]

- 给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在不修改节点内部的值的情况下完成本题（即，只能进行节点交换）。

```
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode* fakehead = new ListNode(0);
        fakehead->next = head;
        ListNode* cur = fakehead;
        while(cur->next!=nullptr&&cur->next->next!=nullptr)
        {
            ListNode* tmp1 = cur->next;
            ListNode* tmp2 = cur->next->next;

            cur->next->next = cur->next->next->next;
            cur->next = tmp2;
            cur->next->next = tmp1;

            cur = tmp1;
        }
        return fakehead->next;
    }
};
```

- 依旧采用经典的假头节点法，避免了头节点的讨论。注意到我们每当面对这种题目时，可以画图，然后改一个指针擦一个箭头画一个箭头，不仅可以弄清楚先后顺序，还可以搞明白哪些指针需要提前存储一个副本。

例题：leetcode 19 - 删除链表的倒数第 N 个结点

[19. 删除链表的倒数第 N 个结点]

- 给你一个链表，删除链表的倒数第 `n` 个结点，并且返回链表的头结点。

```
class Solution {
public:
```

```
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode* fakehead = new ListNode(0);
        fakehead->next = head;

        ListNode* cur = fakehead;
        while(n--) cur=cur->next;
        ListNode* del = fakehead;
        while(cur!=nullptr&&cur->next!=nullptr){
            cur=cur->next;
            del=del->next;
        }
        if(del==nullptr||del->next==nullptr) return nullptr;

        ListNode* tmp = del->next;
        del->next = tmp -> next;
        delete tmp;

        return fakehead->next;
    }
};
```

- 假头节点法的关键：return fakehead->next可以避免很多争端，同时注意特殊情况可以单独考虑。

2.2.3 双指针与链表

例题：面试题 02.07 - 链表相交

[面试题 02.07. 链表相交]

- 给你两个单链表的头节点 `headA` 和 `headB` ，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 `null` 。

```
class Solution {
public:
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        int lena=0,lenb=0;
        ListNode* cur = headA;
        while(cur!=nullptr){
            lena++;
            cur=cur->next;
        }
        cur = headB;
        while(cur!=nullptr){
            lenb++;
            cur=cur->next;
        }

        ListNode* sa=headA,* sb=headB;
        int pre = abs(lena-lenb);

        if(lena>lenb) while(pre--) sa=sa->next;
        if(lena<lenb) while(pre--) sb=sb->next;
```

```

        while(sa!=sb&&sa!=nullptr){
            sa = sa->next;
            sb = sb->next;
        }
        return sa;

    }
};
```

- 这道题在思维上的难度在于如何排查交点：
    - 事实上交点之后所有的项肯定是一致的，所以我们先对齐再查找就行了。

例题：leetcode 142 - 环形链表 II

[142. 环形链表 II]

- 给定一个链表的头节点 `head` ，返回链表开始入环的第一个节点。 *如果链表无环，则返回* `null`。
- 如果链表中有某个节点，可以通过连续跟踪 `next` 指针再次到达，则链表中存在环。 为了表示给定链表中的环，评测系统内部使用整数 `pos` 来表示链表尾连接到链表中的位置（**索引从 0 开始**）。如果 `pos` 是 `-1`，则在该链表中没有环。**注意：`pos` 不作为参数进行传递**，仅仅是为了标识链表的实际情况。
- **不允许修改** 链表。

```
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        unordered_set<ListNode*> st;
        while(head){
            if(st.count(head)) return head;
            else {st.insert(head); head=head->next;}
        }
        return nullptr;
    }
};
```

- 一个简单的想法就是用哈希表去存储节点的地址，一旦有重复就肯定是成环了。
- 这种想法看似简单但空间消耗并非O(1)，我们有另外一个思路如下：
    - 判断链表是否有环：
        - 可以使用快慢指针法，分别定义 fast 和 slow 指针，从头结点出发，fast指针每次移动两个节点，slow指针每次移动一个节点，如果 fast 和 slow指针在途中相遇 ，说明这个链表有环。
        - 有环的话，一定会在环内相遇，且第一次相遇时慢指针一定还未完成一圈，因为若慢指针已经完成一圈了，那快指针至少完成了两圈，则一定在之前相遇过。
    - 假设从头结点到环形入口节点的节点数为x。 环形入口节点到 fast指针与slow指针相遇节点 节点数为y。 从相遇节点再到环形入口节点节点数为 z。
        - 那么相遇时： slow指针走过的节点数为: `x + y`，fast指针走过的节点数： `x + y + n (y + z)`，n为fast指针在环内走了n圈才遇到slow指针， （y+z）为一圈内节点的个数A。

- 因为fast指针是一步走两个节点，slow指针一步走一个节点， 所以fast指针走过的节点数 = slow指针走过的节点数 * 2：
- 因为要找环形的入口，那么要求的是x，将x单独放在左面：`x = n (y + z) - y = (n - 1) (y + z) + z`，
  - 这就意味着，**从头结点出发一个指针，从相遇节点也出发一个指针，这两个指针每次只走一个节点， 那么当这两个指针相遇的时候就是环形入口的节点。**

```
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        ListNode* fast = head;
        ListNode* slow = head;
        while(fast!=nullptr&&fast->next!=nullptr){
            fast=fast->next->next;
            slow=slow->next;
            if(fast==slow){
                ListNode* find=head;
                while(fast!=find){
                    fast=fast->next;
                    find=find->next;
                }
                return find;
            }
        }
        return nullptr;
    }
};
```